Mamba-3 Part 2 - Methodological Deep Dive

AUTHORS
Kevin Y. Li* (bloggin)
Aakash Lahoti*
Berlin Chen*
Caitlin Wang*
Aviv Bick
J. Zico Kolter
Tri Dao
Albert Gu

AFFILIATIONS
CMU
CMU
Princeton
Princeton
CMU
CMU
Princeton, Together AI
CMU, Cartesia AI

PUBLISHED
March 16, 2026

Contents

State Space Foundations
Upgraded Discretization
- Exponential-Adjusted Discretizations
- Recovering Prior Mamba Discretization
- New Exponential-Trapezoidal Discretization
- Parallel Representation of New Recurrence

Complex-valued SSM
- Parity, what is that?
- Representing with Real-valued SSMs
- Efficient Implementation with RoPE Trick

Multi-Input, Multi-Output
- Working while Memory-Bound
- The MIMO System
- Intuition and Training
- Instantiation

The End, For Now

1. Part I
2. Part II

We introduced our Mamba-3 model in Part I in which we mentioned that the three core methodological changes were inspired by the SSM perspective. Here, we'll actually do a deep dive into what each of these three improvements entail, their motivations, and their derivations.

But first, let's refresh our memory on the underlying state space model and its background.

State Space Foundations

The state space model, at its most primitive, is a simple, **continuous** ordinary differential equation (ODE). The input $x(t) \in \mathbb{R}$ is mapped to output $y(t) \in \mathbb{R}$ through a hidden state $h(t) \in \mathbb{R}^N$ of size N, also referred to as the state size. In the past, in both deep learning and classical control theory, these systems were **linear time-invariant** (LTI), where the "state decay" transition $A \in \mathbb{R}^{(N\times N)}$, $B \in \mathbb{R}^N$, and $C \in \mathbb{R}^N$ terms were constant.

$$h'(t) = Ah(t) + Bx(t)$$
$$y(t) = C^\top h(t)$$

We will occasionally refer to A as the *state-transition*, and $Bx(t)$ as the *state-input*.

Upon **discretization** with one's favorite method, as demonstrated with the zero-order hold (ZOH) used in both Mamba-1 and Mamba-2, a familiar **recurrence** materializes,

$$h_t = e^{\Delta_t A_t} h_{t-1} + A_t^{-1}(e^{\Delta_t A_t} - I)\, B_t\, x_t$$
$$y_t = C_t^\top h_t$$

where the discretized $\bar{A}$ and $\bar{B}$ are now $e^{\Delta_t A_t}$ and $A_t^{-1}(e^{\Delta_t A_t} - I)\, B_t$ respectively.

Eagle-eyed readers may ask "how does one go from a LTI system to a linear-time varying (LTV) system?" — if you did, the answer is revealed below!

▶ Aside on prior Mamba discretizations

While there are no theoretical restrictions on the class of matrices $\bar{A}$ can be, **computational constraints keep transition matrices structured**, e.g., diagonal, scalar times identity, Householder (identity plus low-rank), etc.

Great, now we've set up the underlying mechanism of Mamba used in both Mamba-1 and Mamba-2!

As a quick recap, Mamba-3 builds on Mamba-2 to *improve the efficiency-performance trade-off* of current SSMs with inference at the forefront. The three core improvements we've been discussing in this post are rooted in classical state space theory:

1. Instantiating a more generalized recurrence through a formal framework for discretizing the underlying ODE
2. Improving state-tracking abilities by converting to complex-valued SSM without the engineering challenges of explicit complex numbers
3. Increasing the expressivity of the SSM without increasing state size through a multi-input, multi-output (MIMO) formulation.

Let's jump straight into it.

Upgraded Discretization

Our end goal is to obtain a more general recurrence than that of current models from first principles. Luckily for us, the discretization of the continuous ODE provides the *perfect* opportunity to do so.

But first, let's lay down the foundation for our framework used in discretizing time-varying systems. Remember how we mentioned that prior Mamba discretizations adapted the canonical ZOH discretization by adding a time subscript to convert the method from LTI to LTV? Well, to be honest, we left a bit more out earlier. The actual implementation of Mamba frankensteined the canonical ZOH and Euler methods to create discretized parameters $\bar{A}_t = \exp(\Delta_t A_t), \bar{B}_t = \Delta_t B_t$.

Holy heuristic! But it works empirically ¯\(ツ)/¯.

One potential explanation for why this mixture does so well despite not being theoretically grounded is that Euler is an approximation of ZOH. Taking the ZOH formula for discretized $\bar{B}_t = A_t^{-1}\left(\exp(\Delta_t A_t) - I\right) B_t$, if we use the approximation $\exp(x) \approx 1 + x$, the resulting $\bar{B}_t \approx \Delta_t B_t$.

This heuristic was always bugging us in the back of our minds, so in our work, we finally formalized the discretization.

> *We develop a method that produces a class of formal discretizations for time-varying systems, including one called exponential-Euler that exactly corresponds to the formula used in Mamba-1/2.*

Discretization Method	α_t	β_t	γ_t	Appearance
Forward Euler	$I + \Delta A$	—	Δ	—
Backward Euler	$(I - \Delta A)^{-1}$	—	$(I - \Delta A)^{-1}\,\Delta$	—
Trapezoidal	$(I - \frac{\Delta}{2}A)^{-1}\,(I + \frac{\Delta}{2}A)$	—	$(I - \frac{\Delta}{2}A)^{-1}\,\Delta$	S4
Zero-Order Hold	$\exp(\Delta A)$	—	$A^{-1}\,(\exp(\Delta A) - I)$	S4D, S5
Zero-Order Hold	$\exp(\Delta_t A_t)$	—	$A_t^{-1}\,(\exp(\Delta_t A_t) - I)$	Mamba-1, -2
Exponential-Euler	$\exp(\Delta_t A_t)$	—	Δ_t	
Exponential-Trapezoidal	$\exp(\Delta_t A_t)$	$(1 - \lambda_t)\Delta_t\,\exp(\Delta_t A_t)$	$\lambda_t\,\Delta_t$	Mamba-3

Resulting final recurrent $h_t = \alpha_t h_{t-1} + \beta_t B_{t-1} x_{t-1} + \gamma_t B_t x_t$ from various discretization methods. Top half is LTI methods, and bottom is LTV methods derived from our discretization framework.

Exponential-Adjusted Discretizations

So let's actually figure out how to discretize our LTV system in a principled manner.

$$h'(t) = A(t)h(t) + B(t)x(t)$$

The general intuition behind our framework is that a bare-bones ODE $f'(t) = Af(t)$ has a closed-form solution $f(t) = e^{tA}f(0)$. It follows that the one-step discrete update is then $x_{t+1} = e^{\Delta A}x_t$. Here, since the derivative includes $Ah(t)$, the state directly impacts the rate of change. Thus, the parameterization of A can rapidly oscillate the dynamics of the system, which forces explicit methods, like Euler, to take small Δ steps which limits the expressivity of the system.

To mitigate this, we **adjust** the dynamics with an integrating factor of e^{-At} to counteract the dominating exponential and directly analyze $e^{-At}h(t)$ instead. Let's see how it applies to our system.

Taking our $h'(t) = A(t)h(t) + B(t)x(t)$ system, we apply an integrating factor of $e^{\int_0^t -A(s)ds}$ as A is now time-varying.

$$\begin{aligned} e^{\int_0^t -A(s)ds}h'(t) &= e^{\int_0^t -A(s)ds}A(t)h(t) + e^{\int_0^t -A(s)ds}B(t)x(t) \\ (e^{\int_0^t -A(s)ds}h(t))' &= e^{\int_0^t -A(s)ds}B(t)x(t) \end{aligned}$$

since $(e^{\int_0^t -A(s)ds})' = -A(t)e^{\int_0^t -A(s)ds}$.

Thus, when we want to discretize between timesteps $[\tau_{t-1}, \tau_t]$, we can just integrate both sides over that interval. For ease of notation, we denote $z(t) := e^{\int_0^t -A(s)ds}$.

$$\begin{aligned} \tfrac{d}{dt}(z(t)h(t)) &= z(t)B(t)x(t) \\ \int_{\tau_{t-1}}^{\tau_t} \tfrac{d}{d\tau}(z(\tau)h(\tau))d\tau &= \int_{\tau_{t-1}}^{\tau_t} z(\tau)B(\tau)x(\tau)d\tau \\ z(\tau_t)h(\tau_t) - z(\tau_{t-1})h(\tau_{t-1}) &= \int_{\tau_{t-1}}^{\tau_t} z(\tau)B(\tau)x(\tau)d\tau \\ z(\tau_t)h(\tau_t) &= z(\tau_{t-1})h(\tau_{t-1}) + \int_{\tau_{t-1}}^{\tau_t} z(\tau)B(\tau)x(\tau)d\tau \end{aligned}$$

Rearranging into a more familiar form and substituting back the $z(\tau)$ value:

$$\begin{aligned} h(\tau_t) &= z(\tau_t)^{-1}z(\tau_{t-1})h(\tau_{t-1}) + z(\tau_t)^{-1}\int_{\tau_{t-1}}^{\tau_t} z(\tau)B(\tau)x(\tau)d\tau \\ h(\tau_t) &= \exp\left(\int_{\tau_{t-1}}^{\tau_t} A(s)ds\right)h(\tau_{t-1}) + \int_{\tau_{t-1}}^{\tau_t} \exp\left(\int_{\tau}^{\tau_t} A(s)ds\right)B(\tau)x(\tau)d\tau \end{aligned}$$

Now, we've isolated the state-transition and the state-input through our integration factor. This means the most "difficult" part of the adjusted system can be calculated independently of the state-input integral, which is left to be approximated with many possible methods.

Under the LTV case, because $A(s)$ is continuous, we "sample" it with a right-hold assumption where $\forall s \in [\tau_{t-1}, \tau_t], A(s) = A(\tau_t) = A_t$, resulting in

$$h_t \approx \exp(\Delta_t A_t)h_{t-1} + \int_{\tau_{t-1}}^{\tau_t} \exp\left((\tau_t - \tau)A_t\right)B(\tau)x(\tau)d\tau$$

▶ When can the transition integral be directly calculated?

This final equation lays the foundation for recovering the prior Mamba discretization methods and is also the inspiration of our "exponential-" style name, as the application of the integration factor can be seen as a style of exponential tilting or adjustment.

Recovering Prior Mamba Discretization

As previously mentioned, prior Mamba discretizations *differ* on paper and in practice. Now, using our new discretization derivation and certain sampling assumptions, we can recover the reported LTV ZOH discretization *and* the implemented exponential-adjusted Euler discretization scheme, or exponential-Euler for short.

We have already recovered the $\bar{A}$ term for both, so we will focus only on the remaining state-input integral which evaluates to $\bar{B}$.

ZOH: Assuming similar assumptions where $B(\tau), x(\tau)$ are constant and sampled at the right endpoint,

$$\begin{aligned} (\cdot) &= B(\tau_t)x(\tau_t)\int_{\tau_{t-1}}^{\tau_t} \exp\left((\tau_t - \tau)A_t\right)d\tau \\ &= B_t\, x_t \exp(A_t\tau_t)\int_{\tau_{t-1}}^{\tau_t} \exp\left(-A_t\tau\right)d\tau \\ &= B_t\, x_t \exp(A_t\tau_t)\frac{1}{A_t}(\exp(-A_t\tau_{t-1}) - \exp(-A_t\tau_t)) \\ &= A_t^{-1}\left(\exp(A_t(\tau_t - \tau_{t-1})) - I\right)B_t\, x_t \\ &= A_t^{-1}\left(\exp(\Delta_t A_t) - I\right)B_t\, x_t \end{aligned}$$

Exponential-Euler: Once again, we approximate the integral with Euler's rule and hold the B, x terms to the right endpoint.

$$
\begin{aligned}
(\cdot) &= B(\tau_t)x(\tau_t)\int_{\tau_{t-1}}^{\tau_t}\exp\left((\tau_t-\tau)A_t\right)d\tau \\
&= B_t\,x_t\exp(A_t\tau_t)\int_{\tau_{t-1}}^{\tau_t}\exp\left(-A_t\tau\right)d\tau \\
&\approx B_t\,x_t\exp(A_t\tau_t)(\tau_t-\tau_{t-1})\exp\left(-A_t\tau_t\right) \\
&= \Delta_t\,B_t\,x_t
\end{aligned}
$$

New Exponential-Trapezoidal Discretization

Thus, the linchpin of converting continuous-time SSMs into various tangible recurrences is *approximating* $\int_{\tau_{t-1}}^{\tau_t}\exp\left((\tau_t-\tau)A_t\right)B(\tau)x(\tau)d\tau$ *in different ways*. We've shown above that one can either analytically solve it (**ZOH**) or approximate it (**Euler**), but what if we don't want to use inverses but also want something more precise than Euler's?

We can instead use the **trapezoid method** to approximate the integral using both endpoints instead of just one for Euler. Unlike the standard method which averages both endpoints, we use a convex combination which we find empirically performs better. The integral then evaluates to

$$
\begin{aligned}
&\Delta_t\left(\lambda_t\exp((\tau_t-\tau_t)A_t)B_t\,x_t+(1-\lambda_t)\exp((\tau_t-\tau_{t-1})A_t)B_{t-1}\,x_{t-1}\right) \\
=&(1-\lambda_t)\Delta_t\,\exp(\Delta_tA_t)\,B_{t-1}\,x_{t-1}+\lambda_t\,\Delta_t\,B_t\,x_t
\end{aligned}
$$

Interestingly, we can see here that for our new exponential-trapezoidal recurrence, there is some structured time-mixing across the state-input terms. Thus, it acts as an **implicit data-dependent convolution** of size two on the SSM's state-input.

Parallel Representation of New Recurrence

Now how can we format our recurrence in a parallel representation to enable faster training? To do so, we'll be viewing the recurrence in its parallel form. This hearkens back to the state space duality (SSD) framework introduced in Mamba-2.

Let's rewrite our recurrence so this doesn't get *too* messy:

$$
h_t=\alpha_th_{t-1}+\beta_tB_{t-1}x_{t-1}+\gamma_tB_tx_t
$$

where $\alpha_t=e^{\Delta_tA_t},\beta_t=(1-\lambda_t)\Delta_te^{\Delta_tA_t},\gamma_t=\lambda_t\Delta_t$.

▶ Refresher on SSD

If we expand the recurrence where $h_{-1}=0$,

$$
\begin{aligned}
&h_0=\gamma_0B_0x_0 \\
&h_1=(\alpha_1\gamma_0+\beta_1)B_0x_0+\gamma_1B_1x_1 \\
&h_2=\alpha_2(\alpha_1\gamma_0+\beta_1)B_0x_0+(\alpha_2\gamma_1+\beta_2)B_1x_1+\gamma_2B_2x_2 \\
&\dots \\
&h_T=\alpha_{T\dots2}(\alpha_1\gamma_0+\beta_1)B_0x_0+\dots+\gamma_TB_Tx_T
\end{aligned}
$$

, we can express the output as a matrix operation

$$
\begin{bmatrix}y_0\\y_1\\y_2\\\vdots\end{bmatrix}=\left(\begin{bmatrix}\gamma_0 & & & \\(\gamma_0\alpha_1+\beta_1) & \gamma_1 & & \\\alpha_2(\gamma_0\alpha_1+\beta_1) & (\gamma_1\alpha_2+\beta_2) & \gamma_2 & \\\vdots & & & \ddots\end{bmatrix}\odot\begin{bmatrix}C_0^\top B_0 & & & \\C_1^\top B_0 & C_1^\top B_1 & & \\C_2^\top B_0 & C_2^\top B_1 & C_2^\top B_2 & \\\vdots & & & \ddots\end{bmatrix}\right)\begin{bmatrix}x_0\\x_1\\x_2\\\vdots\end{bmatrix}
$$

which can further be decomposed into a 1-semiseparable matrix (Mamba-2's decay mask) and a 2-band matrix.

> *The equivalence established between the recurrent and parallel forms of Mamba-3 is another instance of what Mamba-2's SSD established: that certain classes of SSMs have a matrix form that vectorizes the recurrence. This forms the* ***foundation of the hardware-efficient algorithm*** *used for training.*

Complex-valued SSM

As we've mentioned, the simplification of the SSM over the past couple years to improve efficiency has reduced the abilities of newer models [1]. This has been corroborated by a whole host of work which finds that linear RNN-style models are theoretically constrained on state-tracking tasks by their lack of non-linearity between timesteps and their structured matrix transitions, both of which, unfortunately, are critical to their efficient computation [2] [3] [4] [5].

While more complex state transitions, such as diagonal plus low-rank (DPLR) in older LTI SSM models, can improve the method's expressivity, the simplification of transitions across iterations of LTV SSMs has resulted in even the simplest state-tracking tasks falling out-of-reach for Mamba-style models [2]. The inability to solve some of these state-tracking synthetics may signal poor performance in practice where models might need to keep track of parentheses and diffs for coding or actions and states throughout a story.

Parity, what is that?

One of the simplest tasks, **parity**, determining whether the sum of a sequence of 0's and 1's is even, is unsolvable by Mamba models in a constant number of layers. The ideal solution requires the hidden state to track whether the running sum is even or odd and to alternate depending on the next input, modeling a simple two-state automaton [2]. While this seems simple enough, current Mamba models constrain the transition $\bar{A}_t \in [0, 1]$, which *forces* the model to learn the naive solution: add all the values together then mod 2 [3]. This does work for shorter sequences but quickly becomes infeasible when the sequence outgrows the state.

But, parity and other modulo tasks can be solved with **rotations**! The way one can visualize how rotations solve modulo m problems is that one has some 2D vector that can be rotated around the origin. The entire possible angle distribution $[0, 2\pi]$ is partitioned into m sections, and the vector is rotated by $\frac{2\pi}{m}$ to align with the current running modulo remainder.

Representing with Real-valued SSMs

Working with complex values in computer systems is quite a pain due to their multiplicative interactions. Luckily for us, **diagonal complex-valued continuous SSMs *can* be represented as discretized real-valued SSMs** (without any additional approximation loss compared to standard discretization).

While the full proof can be found within our paper, the general intuition is to expand the original N-sized state complex-valued SSM to a $2N$-sized real-valued where each complex-valued dimension is split into its real and imaginary counterpart, and the complex transition matrix is partitioned into its scaling and oscillatory portions. With the commutative property of the scaling and oscillatory components in effect (due to the diagonal structure of the underlying matrix), we can map the continuous diagonal complex transition into a block-diagonal, scaled rotation transformation.

▶ Converting general complex SSMs to real-valued SSMs

Eventually, under the prior exponential-Euler discretization, we obtain the following recurrence

$$h_t = e^{\Delta_t A_t} \underbrace{\begin{bmatrix} \cos(\Delta_t \theta_t) & -\sin(\Delta_t \theta_t) \\ \sin(\Delta_t \theta_t) & \cos(\Delta_t \theta_t) \end{bmatrix}}_{R_t} h_{t-1} + \Delta_t B_t x_t$$

for an $N = 2$ state. For larger states, the rotation matrix R_t is block-diagonal and the θ's can differ.

Efficient Implementation with RoPE Trick

Great, now we've shown that we can implement a complex SSM without having to explicitly model the imaginary components! But another issue remains: the rotation of the hidden state requires us to reimplement the kernels to incorporate this new type of transition — more moving parts, rotating the entire hidden state, etc., — seems like quite the hassle *ugh*... or is it?

Luckily for us, given the structure of A, we can sidestep all of this and directly adjust the B, C to achieve the same goal. This is because the output for timestep t can be modeled as

$$y_t = C_t^\top \bar{B}_t + \cdots + C_t^\top (\bar{A}R)^{\times}_{t:1} \bar{B}_0$$

Since $\bar{A}$ is a scaled identity matrix, we can ignore the $\bar{A}$ terms for now by absorbing them into C. This results in the term $C_i^\top R_i \cdots R_{j+1} \bar{B}_j$ which can be represented by $(R_i \cdots R_0 C_i)^\top (R_j \cdots R_0 \bar{B}_j)$. The $\bar{A}$ terms can be reintroduced at this point. Thus, it's apparent that the rotations can be embedded into the B, C terms prior to performing the SSM recurrence instead of directly adjusting the transition matrix.

> *The application of our data-dependent rotations onto B, C can be done efficiently. Instead of performing numerous matrix multiplications, we can run a cumulative sum over the θ's and perform the efficient realization of rotary matrix multiplication from the RoFormer paper [7], which itself used data-independent rotations. This inspired us to call the use of a vanilla SSM to compute a complex SSM as the "RoPE trick."*

The RoPE trick extends to our exponential-trapezoidal recurrence, and we empirically validate that our complex-valued SSM is able to solve state-tracking tasks previously too hard for prior Mambas.

Model	Parity ↑	Arith. w/o brackets ↑	Arith. w/ brackets ↑
Mamba-3	100.00	98.51	87.75
Mamba-3 (w/ Std. RoPE)	1.56	20.70	2.62
Mamba-3 (w/o RoPE)	2.27	1.49	0.72
Mamba-2	0.90	47.81	0.88
GDN [-1,1]	100.00	99.25	93.50

Converting the SSM from real to complex-valued gives the model the capability to solve parity and other state-tracking tasks.

Multi-Input, Multi-Output

The compute paradigm in scaling LLMs has shifted from training to inference in the past two years or so. Nowadays, more and more compute is dedicated to the actual deployment and usage of these models, and to some degree, the writing was on the wall. Emergent properties such as chain-of-thought and in-context learning techniques dramatically improved performance of earlier models by making them think longer and process more tokens, and all of the best models to date are reasoners which have (almost certainly) been post-trained with reinforcement learning using large rollout budgets. With the advent of agentic workflows, we have agents spawning subagents and so forth.

> *What does such a paradigm change mean for hardware efficiency?*

Working while Memory-Bound

Compared to the **compute-bound** nature of training, the deployment of the same models, especially decoding, is **memory-bound**. Throughout training, the hardware is constantly performing operations, but during decoding, the compute units of the *hardware sit idle* for large swathes of time as it waits for data to be moved across different levels of memory hierarchy.

As an example of why this happens, think about a simple MLP. During training, the entire sequence is processed, but during decode, only the current token is processed as the past tokens are cached. The latency spent moving the MLP weights is around the same for both training and decode, but can be amortized a lot better over more computation under a training regime.

So with current linear models where the state update and output calculation can be performed in **constant time**, compute units sit idle for most of the time, and we are bottlenecked by simply moving data back and forth! One way to estimate how "hard" the hardware is working is through **arithmetic intensity**, a ratio of compute performed to memory moved.

Let's analyze how SSMs are deployed in practice and their arithmetic intensity. A typical SSM, say Mamba-2 for instance, is organized into heads with head dimension P, where a single head is composed of P SISO SSMs that share the same a_t, B_t, C_t, B_t

$$\begin{aligned}\mathbf{h}_t &= a_t \mathbf{h}_{t-1} + B_t \mathbf{x}_t \\ \mathbf{y}_t &= C_t^\top \mathbf{h}_t\end{aligned}$$

where a_t is a scalar decay and $\mathbf{x}_t, \mathbf{y}_t \in \mathbb{R}^P, \mathbf{h}_t \in \mathbb{R}^{N \times P}$. If we use 2-byte data, for a single decode step, the total memory traffic is $2(1 + 2N + P + NP)$ when accounting for all SSM parameters. The movement of the hidden state is clearly the main contributing factor at reasonable values of P and N.

When calculating the number of FLOPs used for the same operation, we get around $5NP - P$ [4]. Thus, default SSM decoding has an arithmetic intensity of around 2.5. To put this into context, the arithmetic intensity of matmuls for an H100 is around 300 ops per byte; anything above this is compute-bound. Having an arithmetic intensity of as low as 2.5 means that decoding is squarely memory-bound… *yikes*

Since we pay for the entire rack and the expensive tensor cores, how can we use as many of them as possible?

The MIMO System

▶ Q: How does one increase a ratio?

We've seen empirically that the state size is quite important for performance but expanding it also increases memory… so, let's keep that the same. Now how do we increase the compute required for calculating the hidden state recurrence while maintaining the same hidden state? Referring back to our state space/control theory toolbox, **multi-input, multi-output (MIMO)** SSMs can be used instead of the single-input, single-output (SISO) SSMs we've been using.

Through the expansion of the dimension of $\mathcal{C}_t, \mathcal{B}_t$ to $N \times R$ and $\mathbf{x}_t, \mathbf{y}_t$ to $P \times R$ where R is the rank of the system, we can maintain similar memory traffic (for small enough R) while increasing the FLOPs utilized when operating with matrix multiplications with $B_t x_t^\top$ instead of outer-products.

$$\begin{aligned}\mathbf{h}_t &= a_t h_{t-1} + \mathcal{B}_t \mathbf{x}_t^\top \\ \mathbf{y}_t &= \mathcal{C}_t^\top \mathbf{h}_t\end{aligned}$$

The total FLOP count thus increases to $4NPR + NP - PR$ which **results in an arithmetic intensity that scales with** $O(R)$ when $R \ll P, N$ (generally the case as $P = 64, N = 128$ and $R = 4$).

Intuition and Training

> *The downstream gains and comparable decoding latency associated with switching from SISO to MIMO require compute costs that scale linearly with R during compute-bound training.*

Expressing the output of a MIMO SSM would require R^2 SISO SSMs due to its rank R state-input and R unique outputs. Its hidden state can be partitioned into the sum of R SISO hidden states, and subsequently, the hidden state needs to be instantiated R times for each of the outputs. But if the expressivity is a R^2 increase, how does training compute required scale by *only* R?

The **chunked training algorithm** is the reason for this disparity. Most linear models, including both Mamba-2 and Mamba-3, are computed in chunked fashion, where the sequence is partitioned into chunk sizes of C. The hidden state is aggregated across chunks in a *sequential* manner, while the output of the SSM is calculated with a quadratic, *parallel* algorithm.

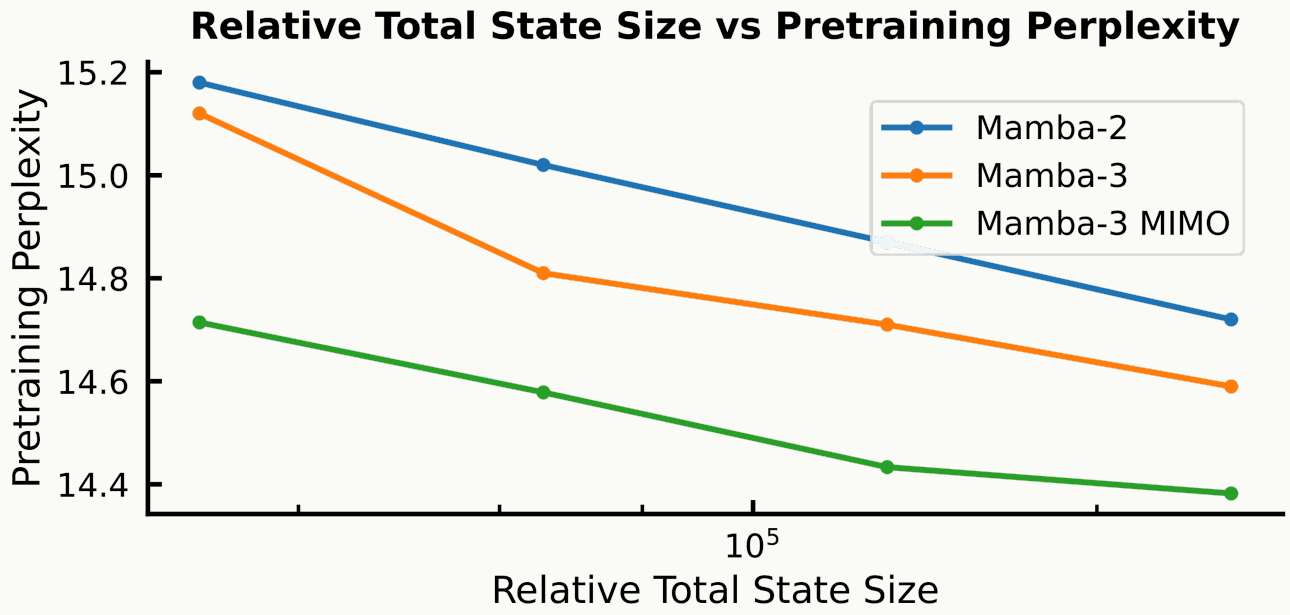
For MIMO, the computation of outputs between chunks increases by a factor of R, whereas the computation of outputs within each chunk increases by R^2. So by decreasing the chunk size to $\frac{C}{R}$, the total FLOP count only increases by a factor of R. Our paper covers the actual FLOP calculations, but one way to think about it is that we want to reduce the amount of compute required for each quadratic algorithmic pass by increasing the number of times we call it.

Instantiation

Given the interpretation of MIMO SSMs as multiple SISO ones, improvements introduced for vanilla SSMs, like our exponential-trapezoidal discretization and complex-valued transition, can be directly applied to our MIMO variant. However, the conversion must be done carefully without drastically increasing the total parameter count. The naive solution of expanding the projection size would lead to a $R\times$ increase as the SSM inputs x, B, C would all need to be adjusted. The subsequent rank R output Y would also force the output gate Z and output projection to expand as well. This approach is clearly untenable.

Instead, we can use Mamba's multi-value attention structure to our advantage. Since the B, C are tied across all heads, we can increase the projection size without much issue, resulting in a fairly negligible $DN \to DNR$ increase for the entire layer. However, the input x, output y, and gate Z are unique per head and are the main source of parameters, thus cannot be increased in such a way. Instead, we keep the original projections then element-wise scale each dimension of the projected value to size R using a learnable, data-independent vector. For each head, we are able to reduce the parameter count from DPR to $DP + PR$, which is quite the reduction given the number of heads each Mamba layer has!

We show that our instantiation **balances the expressivity of multi-input, multi-output systems and parameter efficiency**. In parameter-matched settings, our Mamba-3 MIMO variant further improves the already strong performance of regular Mamba-3 at all scales we tested on. When analyzing state size (proxy for decoding speed) to performance in controlled experiments, Mamba-3 sets the Pareto front compared to prior Mamba-2, able to achieve comparable performance with half the state size.



When analyzing the Pareto frontier of state size (a good proxy for decoding speed) to performance, Mamba-3 dominates prior Mamba-2. The MIMO variant of Mamba-3 pushes performance further without increasing state size at all.

> *Mamba-3 offers a faster model with the same quality or a better model for the same speed.*

The End, For Now

We had to cut a bunch of proofs and results to keep the content in here digestible, but if that interests you, please do read our paper!

Within our work, we've aimed at boosting the performance and capabilities of the Mamba series from a few SSM-centric improvements. We're curious to see how and where the community explores in architecture research. In particular, we are quite excited (and think addressing them would be really impactful) in the following directions:

- **Building better hybrids**: It's been amazing to see the general research community and industry labs appreciate the benefits linear models can provide, especially with hybrid models [6] [8] [9]. Most architectures follow an interleaved structure, but the "science" of what enables good linear-self attention synergy is still unknown. We've seen a lot of cool work making important ground, e.g., shifting from RoPE to NoPE for attention layers [10] or keeping the first and last layers attention-free [11] A more meta question might be: *are interleaved hybrid models truly the best way to utilize linear models?*
- **Improving Layer Primitives**: Our methodological improvements, while most natural to SSMs, can be applied to other architectures. It would be interesting to see how they scale under different transition mechanisms. In addition, there seems to be a whole trove of untapped improvements waiting to be uncovered or inspired in the "classics," if you will. Just as Mamba and other SSMs are grounded within signal processing and traditional state space literature, such parallels can be found in other types of linear models — fast-weight programmers for linear attention [12], for example. *What might the standard transition look like for the best self-attention alternative in two, three years?*

Footnotes

1. We've mainly been highlighting the simplification of diagonal to identity times scalar across the LTV systems of Mamba-1 to Mamba-2, but the original LTI SSMs were actually complex-valued! Mamba-1 simplified the SSM to be all real-valued which empirically did not impact language modeling, but as we will see, reduced state-tracking capabilities. [↩]
2. Related delta-rule based linear attention models, e.g., GDN, KDA [6], are able to partially mitigate these state-tracking limitations through more expressive state transitions, i.e., identity plus low-rank. [↩]
3. If $\bar{A}_t$ can be -1, it would enable the alternating solution, but would require the underlying implementation to be rewritten as currently the $\bar{A}$ is handled in log-space. [↩]
4. A quick rundown why. The $a_t h_{t-1}$ scaling, $B_t x_t$ outer product, and their summation take a total of 3NP. The matmul between $C_t^\top h_t$ takes $2N-1$ per P dimension (N multiplication and $N-1$ accumulation), resulting in a final $5NP-P$. [↩]

References

1. **Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention** [PDF]
   Katharopoulos, A., Vyas, A., Pappas, N. and Fleuret, F., 2020.

2. **Unlocking State-Tracking in Linear RNNs Through Negative Eigenvalues** [PDF]
   Grazzi, R., Siems, J., Zela, A., Franke, J.K.H., Hutter, F. and Pontil, M., 2025.

3. **The Illusion of State in State-Space Models** [PDF]
   Merrill, W., Petty, J. and Sabharwal, A., 2025.

4. **Theoretical Foundations of Deep Selective State-Space Models** [PDF]
   Cirone, N.M., Orvieto, A., Walker, B., Salvi, C. and Lyons, T., 2025.

5. **The Expressive Capacity of State Space Models: A Formal Language Perspective** [link]
   Hahn, M., Sarrof, Y. and Veitsman, Y., 2024. Advances in Neural Information Processing Systems 37, pp. 41202–41241. Neural Information Processing Systems Foundation, Inc. (NeurIPS). DOI: 10.52202/079017-1304

6. **Kimi Linear: An Expressive, Efficient Attention Architecture** [PDF]
   Team, K., Zhang, Y., Lin, Z., Yao, X., Hu, J., Meng, F., Liu, C., Men, X., Yang, S., Li, Z., Li, W., Lu, E., Liu, W., Chen, Y., Xu, W., Yu, L., Wang, Y., Fan, Y., Zhong, L., Yuan, E., Zhang, D., Zhang, Y., Liu, T.Y., Wang, H., Fang, S., He, W., Liu, S., Li, Y., Su, J., Qiu, J., Pang, B., Yan, J., Jiang, Z., Huang, W., Yin, B., You, J., Wei, C., Wang, Z., Hong, C., Chen, Y., Chen, G., Wang, Y., Zheng, H., Wang, F., Liu, Y., Dong, M., Zhang, Z., Pan, S., Wu, W., Wu, Y., Guan, L., Tao, J., Fu, G., Xu, X., Wang, Y., Lai, G., Wu, Y., Zhou, X., Yang, Z. and Du, Y., 2025.

7. **RoFormer: Enhanced Transformer with Rotary Position Embedding** [PDF]
   Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B. and Liu, Y., 2023.

8. **Qwen3 Technical Report** [PDF]
   Team, Q., 2025.

9. **Hunyuan-TurboS: Advancing Large Language Models through Mamba-Transformer Synergy and Adaptive Chain-of-Thought** [PDF]
   Team, T.H., Liu, A., Zhou, B., Xu, C., Zhou, C., Zhang, C., Xu, C., Wang, C., Wu, D., Wu, D., Jiao, D., Du, D., Wang, D., Zhang, F., Lian, F., Xu, G., Zhang, G., Wang, H., Luo, H., Hu, H., Xu, H., Wu, J., Zhu, J., Yan, J., Zhu, J., Zhang, J., Xue, J., Xia, J., Zheng, J., Liu, K., Zhang, K., Zheng, K., Li, K., Wang, K., Jiang, L., Liu, L., Wu, L., Huang, M., Yu, P., Wang, P., Wang, Q., Xiang, Q., Liu, Q., Sun, Q., Guo, R., Xie, R., Yang, S., Chen, S., Hu, S., Li, S., Li, S., Chen, S., Zheng, S., Yang, T., Zhang, T., Yu, T., Han, W., Liu, W., Zhou, W., Wang, W., Chen, W., Feng, X., Ren, X., Sun, X., Kuang, X., Huang, X., Cao, X., Chen, Y., Du, Y., Yang, Z., Tao, Y., Deng, Y., Shen, Y., Hong, Y., Chen, Y., Huang, Y., Deng, Y., Mao, Y., Wang, Y., Zeng, Y., Xu, Z., Kang, Z., Zhao, Z., Yan, Z., Fang, Z., Hu, Z., Chen, Z., Li, Z., Li, Z., Yan, A., Liang, A., Liu, B., Pan, B., Xing, B., Wu, B., Qu, B., Ni, B., Wu, B., Li, C., Jiang, C., Zhang, C., Liu, C., Yang, C., Xu, C., Wang, C., Zha, C., Yi, D., Wang, D., Lu, F., Chen, F., Liu, F., Zheng, F., Yu, G., Li, G., Wang, G., Lin, H., Liu, H., Wang, H., Fei, H., Lu, H., Jiang, H., Sun, H., Zhu, H., Dai, H., Chen, H., Feng, H., Cai, H., Peng, H., Lv, J., Shi, J., Bu, J., Li, J., Hu, J., Guan, J., Xu, J., Cai, J., Zhang, J., Song, J., Jiang, J., Liu, J., Yang, J., Zhang, J., lv, J., Zhao, J., Li, J., Liu, J., Zhao, J., Guo, J., Wang, K., Wu, K., Fu, L., He, L., Wang, L., Liu, L., Dong, L., Zhan, L., Cheng, L., Xu, L., Zheng, M., Liu, M., Hu, M., Chen, N., Chen, P., He, P., Pan, P., Wei, P., Yang, Q., Yi, Q., Wang, R., Chen, R., Sun, R., Yang, R., Chen, R., Zhou, R., Zhang, S., Zhang, S., Xu, S., Chang, S., Liu, S., Wang, S., Feng, S., Yuan, S., Zhang, T., Lang, T., Li, T., Deng, W., Li, W., Wang, W., Zhang, W., Sun, W., Ouyang, W., Jiao, W., Sun, W., Jia, W., Zhang, X., He, X., Ren, X., Zhu, X., Guo, X., Li, X., Ma, X., Lu, X., Feng, X., Huang, X., Guan, X., Li, X., Zhang, X., Gao, X., Luo, X., Qi, X., Chen, Y., Tao, Y., Xiao, Y., Mai, Y., Chen, Y., Ding, Y., Yang, Y., Song, Y., Yang, Y., Zhu, Y., Wu, Y., Liu, Y., Yang, Y., Cai, Y., Tu, Y., Zhang, Y., Huang, Y., Zhou, Y., Jiang, Y., Liu, Y., Hu, Y., Lin, Y., Yang, Y., Wang, Y., Zhang, Y., Wu, Z., Zhang, Z., Yu, Z., Yang, Z., Zhao, Z., Li, Z., Huang, Z., Liu, Z., Xu, Z., Kui, Z., Zeng, Z., Xiong, Z., Han, Z., Wu, Z., Geng, Z., Zhao, Z., Tang, Z., Zhu, Z., Zhu, Z. and Xu, Z., 2025.

10. **Rope to Nope and Back Again: A New Hybrid Attention Strategy** [PDF]
    Yang, B., Venkitesh, B., Talupuru, D., Lin, H., Cairuz, D., Blunsom, P. and Locatelli, A., 2025.

11. **An Empirical Study of Mamba-based Language Models** [PDF]
    Waleffe, R., Byeon, W., Riach, D., Norick, B., Korthikanti, V., Dao, T., Gu, A., Hatamizadeh, A., Singh, S., Narayanan, D., Kulshreshtha, G., Singh, V., Casper, J., Kautz, J., Shoeybi, M. and Catanzaro, B., 2024.

12. **Linear Transformers Are Secretly Fast Weight Programmers** [PDF]
    Schlag, I., Irie, K. and Schmidhuber, J., 2021.

© Copyright 2026 Goomba AI Lab.